

复旦大学计算与智能创新学院 2025-2026 学年面向对象程序设计模拟试卷 02

课程名称：面向对象程序设计

考试形式：闭卷

考试时间：120 分钟

一、选择题（每题只选择一个答案；如选多个，不计分。每题 3 分，共 24 分）

1. 关于模板中的嵌套类型名，下列写法正确的是：

```
template <class T>
T median(std::vector<T> v)
{
    _____
    vec_sz n = v.size();
    return v[n / 2];
}
```

- A. typedef std::vector<T>::size_type vec_sz;
- B. typedef typename std::vector<T>::size_type vec_sz;
- C. typename std::vector<T>::size_type vec_sz;
- D. typedef class std::vector<T>::size_type vec_sz;

2. 下列哪一个运算符重载必须写成成员函数？

- A. operator+
- B. operator<<
- C. operator[]
- D. operator<

3. 若类 Num 定义如下：

```
class Num {
public:
    explicit Num(int x = 0);
};
```

则下列语句中不能通过编译的是：

- A. Num a;
- B. Num b(3);
- C. Num c = Num(5);
- D. Num d = 7;

4. 关于纯虚函数、抽象类和动态绑定，下列说法错误的是：

- A. 含有纯虚函数的类一定是抽象类
- B. 抽象类不能定义对象，但可以定义其指针或引用
- C. 通过基类引用调用虚函数时，是否发生动态绑定取决于引用在定义处的静态类型
- D. 抽象类中可以有数据成员、构造函数和已定义的普通成员函数

5. 关于 vector、list 与标准库算法，下列说法正确的是：

- A. list 支持下标运算，因此可以写 lst[0]
- B. std::sort 既可以作用于 vector，也可以直接作用于 list
- C. copy(v.begin(), v.end(), back_inserter(lst)) 可以把 vector 中的元素追加到 list 末尾
- D. remove_if 调用后一定会真正缩短容器长度

6. 一个类在构造函数中申请动态内存，并在析构函数中释放。关于该类的赋值运算符，下列说法正确的是：

- A. 可以直接把右操作数的指针地址赋给左操作数的指针成员，最省事也最安全
- B. 自我赋值检查没有必要，因为 x = x 根本不会发生
- C. 若不重新申请新空间而直接共享底层指针，常会导致重复释放等问题
- D. 赋值运算和复制构造完全相同，不需要分开考虑

7. 设有如下代码：

```
std::unique_ptr<int> p(new int(10));  
std::unique_ptr<int> q = _____;
```

要使代码通过编译，并让 q 接管资源，横线处应填：

- A. p
- B. *p
- C. std::move(p)
- D. &p

8. 下面关于继承和对象切片的说法中，正确的是：

- A. 把派生类对象赋值给基类对象变量时，派生类部分可能被切掉
- B. 若基类析构函数不是虚函数，则派生类一定不能定义析构函数
- C. 对象切片只会发生在指针和引用上，不会发生在按值传递和按值保存中
- D. 只要发生了继承，程序里所有函数调用都会自动动态绑定

二、程序阅读题（每题 5 分，共 20 分。说明：程序中省略了必要的头文件和 using 声明）

1. 写出下面程序的运行结果。

```
class Base {  
    char b;
```

```

public:
    Base(char c = 'A') : b(c) { cout << "+" << b << " "; }
    virtual ~Base() { cout << "-" << b << " "; }
};

class Derived : public Base {
    Base member;
    char d;
public:
    Derived(char c = 'X') : Base(c + 1), member(c + 2), d(c) {
        cout << "*" << d << " ";
    }
    ~Derived() { cout << "/" << d << " "; }
};

int main()
{
    Base* p = new Derived('M');
    delete p;
    return 0;
}

```

2. 写出下面程序的运行结果。

```

class Counter {
    int value;
public:
    Counter(int v = 0) : value(v) {}

    Counter& operator++() {
        value += 2;
        return *this;
    }

    Counter operator++(int) {
        Counter old(*this);
        value += 3;
        return old;
    }

    friend ostream& operator<<(ostream& os, const Counter& c) {
        os << c.value;
        return os;
    }
}

```

```

    }
};

int main()
{
    Counter c(1);
    cout << c++ << " ";
    cout << ++c << " ";
    cout << c << endl;
    return 0;
}

```

3. 写出下面程序的运行结果。

```

template <class Iter>
int sum_square(Iter begin, Iter end)
{
    int sum = 0;
    while (begin != end) {
        sum += (*begin) * (*begin);
        ++begin;
    }
    return sum;
}

int main()
{
    list<int> nums;
    for (int i = 1; i <= 4; ++i)
        nums.push_back(i);

    cout << sum_square(nums.begin(), nums.end()) << endl;
    return 0;
}

```

4. 写出下面程序的运行结果。

```

class A {
public:
    virtual const char* name() const { return "A"; }
    int value() const { return 1; }
};

```

```

class B : public A {
public:
    const char* name() const { return "B"; }
    int value() const { return 2; }
};

int main()
{
    B b;
    A a = b;
    A& r = b;
    cout << a.name() << a.value() << " ";
    cout << r.name() << r.value() << endl;
    return 0;
}

```

三、程序填空题（每空 3 分，共 24 分。说明：程序中省略了必要的头文件和 using 声明）

1. 补全下面的模板函数 median，使其能够返回 vector<T> 的中值。若向量为空，则抛出异常。

```

template <class T>
T median(std::vector<T> v)
{
    typedef _____(1)_____ vec_sz;
    vec_sz n = v.size();

    if (n == 0)
        throw _____(2)_____;

    std::sort( _____(3)_____ );

    vec_sz mid = n / 2;
    return n % 2 == 0 ? _____(4)_____ : v[mid];
}

```

2. 补全下面的字符串包装类，使其支持 +=、+ 和输出运算。

```

class Str {
public:
    Str(const string& s = "") : data(s) {}

    _____(5)_____ operator+=(const Str& rhs)
    {

```

```

        data += rhs.data;
        return *this;
    }

    _____(6)_____ operator+(const Str& lhs, const Str& rhs)
    {
        Str ret = lhs;
        _____(7)_____;
        return ret;
    }

    friend ostream& operator<<(ostream& os, const Str& s)
    {
        os << s.data;
        _____(8)_____;
    }

private:
    string data;
};

```

四、编程题 (共 32 分)

1. (12 分) 编写模板函数 `intersection_unique(first1, last1, first2, last2, dest)`。

已知两个区间 `[first1, last1)` 和 `[first2, last2)` 中的元素都按非降序排列。请编写模板函数 `intersection_unique`，将两个区间中**同时出现**的元素按非降序写入目标位置 `dest`，并满足以下要求：

1. 若某个值在任一输入区间中重复出现多次，输出中仍只保留一次；
2. 函数返回写入结束位置对应的输出迭代器；
3. 不得调用标准库中现成的集合求交算法；
4. 你的实现应适用于 `vector`、`list` 等容器的迭代器。

以下为测试程序片段：

```

int main()
{
    vector<int> a = {1, 2, 2, 3, 5, 8, 8, 9};
    list<int> b = {0, 2, 2, 4, 5, 8, 10};
    vector<int> c;

    intersection_unique(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));

    for (vector<int>::const_iterator it = c.begin(); it != c.end(); ++it)
        cout << *it << " ";
}

```

```

    cout << endl;
    return 0;
}

```

该程序的输出为：

2 5 8

2. (20 分) 按要求实现抽象基类 Employee、派生类 Manager 和 Programmer，以及管理多个雇员对象的类 Company。

要求如下：

1. 基类 Employee 包含雇员姓名 name，并声明以下成员：
 - 构造函数 Employee(const string& n = "")
 - 虚析构函数
 - 纯虚函数 double income() const
 - 纯虚函数 void raise(double factor)，表示把与工资直接相关的基础部分乘以 factor
 - 纯虚函数 Employee* clone() const
 - 虚函数 void print(ostream& os) const
2. 派生类 Manager：
 - 数据成员：salary、bonus
 - 构造函数：Manager(const string& n = "", double s = 0, double b = 0)
 - 收入 income() 为 salary + bonus
 - raise(factor) 只使 salary 乘以 factor
3. 派生类 Programmer：
 - 数据成员：hourly_wage、hours
 - 构造函数：Programmer(const string& n = "", double w = 0, int h = 0)
 - 收入 income() 为 hourly_wage * hours
 - raise(factor) 只使 hourly_wage 乘以 factor
4. 类 Company 用 vector<Employee*> 保存多个雇员，并支持：
 - 默认构造函数
 - 复制构造函数，要求深拷贝
 - 赋值运算符，要求深拷贝并正确处理自我赋值
 - 析构函数，负责释放所有动态对象
 - void add(const Employee& e)：将一个雇员复制后加入容器
 - void raise_all(double factor)：将所有雇员按同样比例加薪
 - double total_income() const：返回所有雇员收入之和
 - 友元输出运算符 operator<<，按加入顺序逐行输出每个雇员的姓名和收入
5. 可以自行补充必要的辅助函数。

测试程序如下：

```

int main()
{
    Company c;

```

```

Manager m("Wang", 8000, 1200);
Programmer p("Li", 150, 40);

c.add(m);
c.add(p);

Company d = c;
d.raise_all(1.1);

cout.setf(ios::fixed);
cout.precision(2);

cout << c.total_income() << endl;
cout << d.total_income() << endl;
cout << d;
return 0;
}

```

程序的输出为:

```

15200.00
16600.00
Wang 10000.00
Li 6600.00

```